

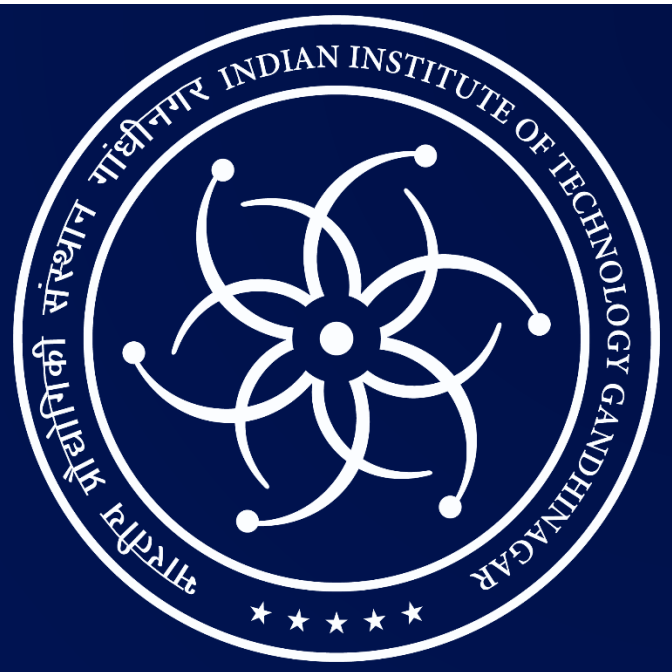


Building DVFS, a Distributed ‘Cloud’ for our Campus

Romit Devendra Mohane
Artificial Intelligence
IIT Gandhinagar

Umang Shikarvar
Computer Science and Engineering
IIT Gandhinagar

Prof. Abhishek Bichhawat
Computer Science and Engineering
IIT Gandhinagar



Abstract

Distributed Virtual File System (DVFS) is a multi-user file system built with golang and gRPC with a dedicated MetaServer for routing and FileServers for data and metadata operations. The system combines server-side inode/ACL metadata with client-side cache trees and callback-based invalidation to provide responsive interactive commands (*ls*, *cd*, *read*, *upload*, *download*, *sharewith*, etc.) while preserving correctness under sharing and concurrent updates. The design emphasizes explicit protocol contracts, persistence with atomic updates, and practical consistency through push notifications plus user-controlled refresh.

Problem Statement and Goals

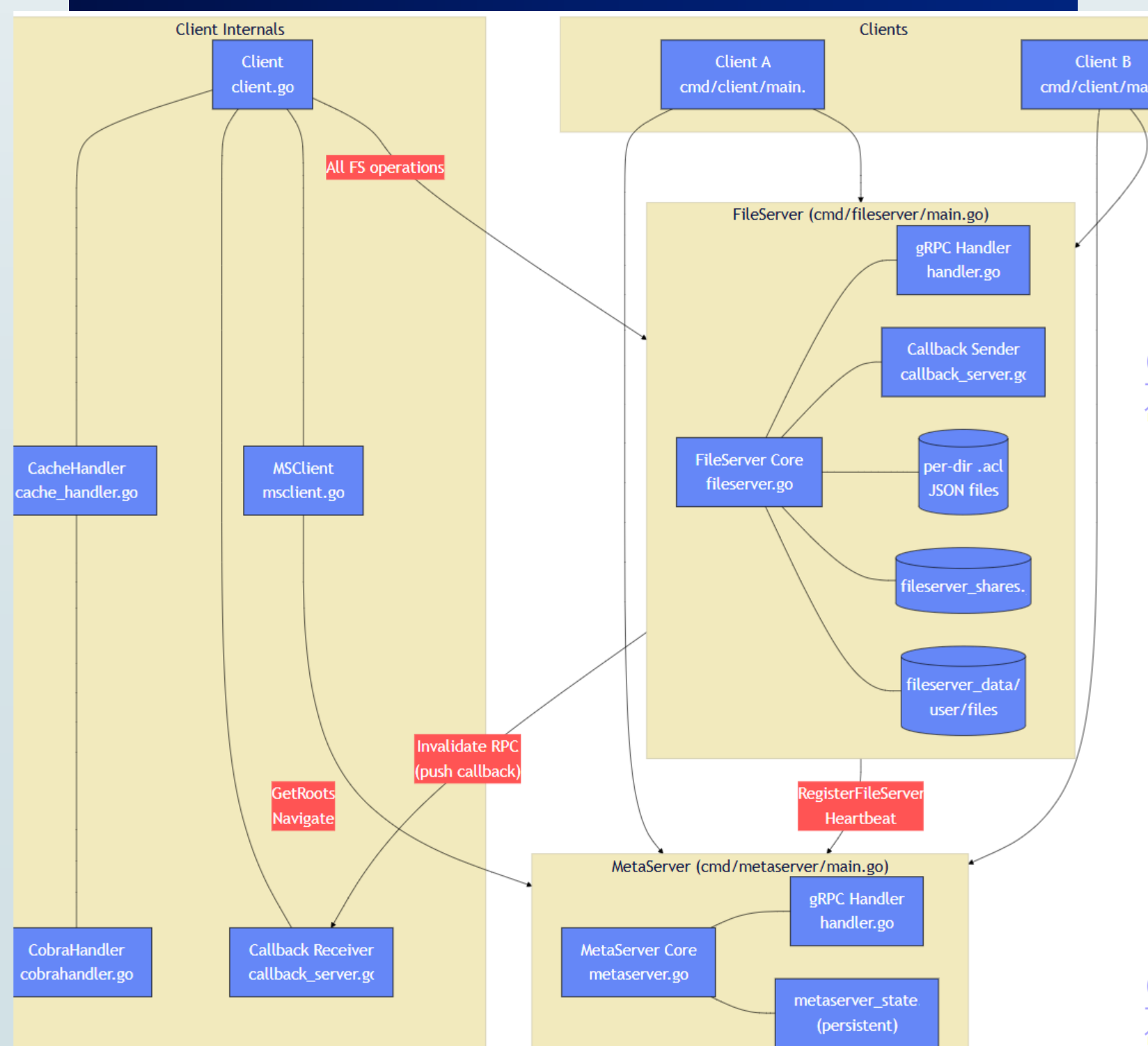
Problem

- Campus teams need a shared storage system that feels as simple as local CLI file operations, while still handling multi-user collaboration, access control, and concurrent updates across labs and personal machines.

Goals

- Build a campus-wide distributed cloud storage layer with intuitive shell-like commands.
- Support secure multi-user sharing with ACL-based control.
- Minimize latency via client-side metadata/content cache.
- Preserve cross-client consistency using server push invalidation callbacks.
- Keep metadata durable across restarts using atomic persistence.

System Architecture



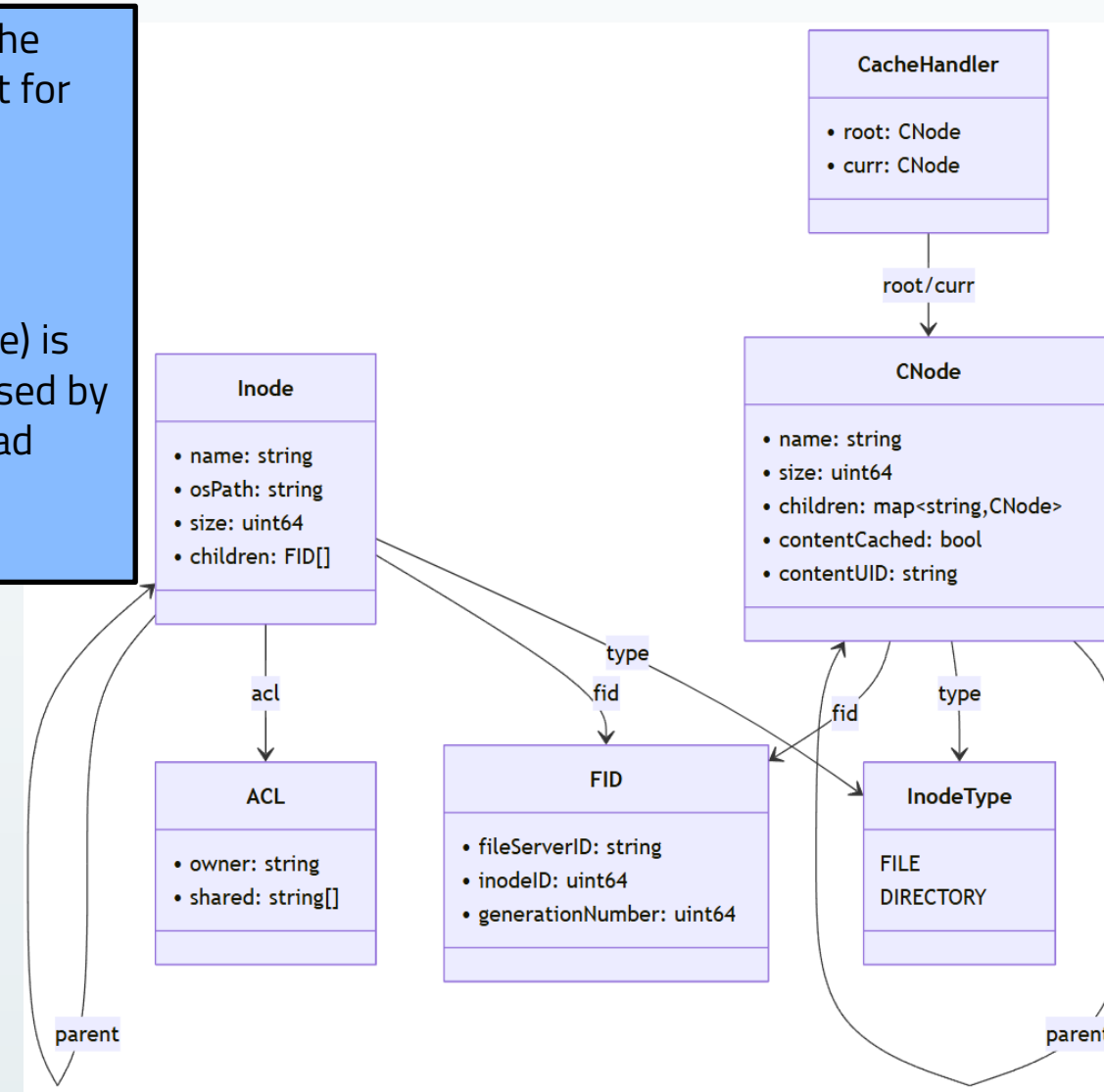
Architecture Highlights

- Control plane:** MetaServer handles user-to-fileserver routing, sharing and health.
- Data plane:** FileServer performs file metadata and content operations.
- Consistency plane:** callbacks invalidate stale cache entries for active users in the same directory.

Data Structures

Inode (server-side inode) is the authoritative metadata object for each file/directory, including identity, ACL, hierarchy, and storage path.

CNode (client-side cache node) is the lightweight local mirror used by the CLI for fast navigation/read decisions and caching with callback-driven invalidation.



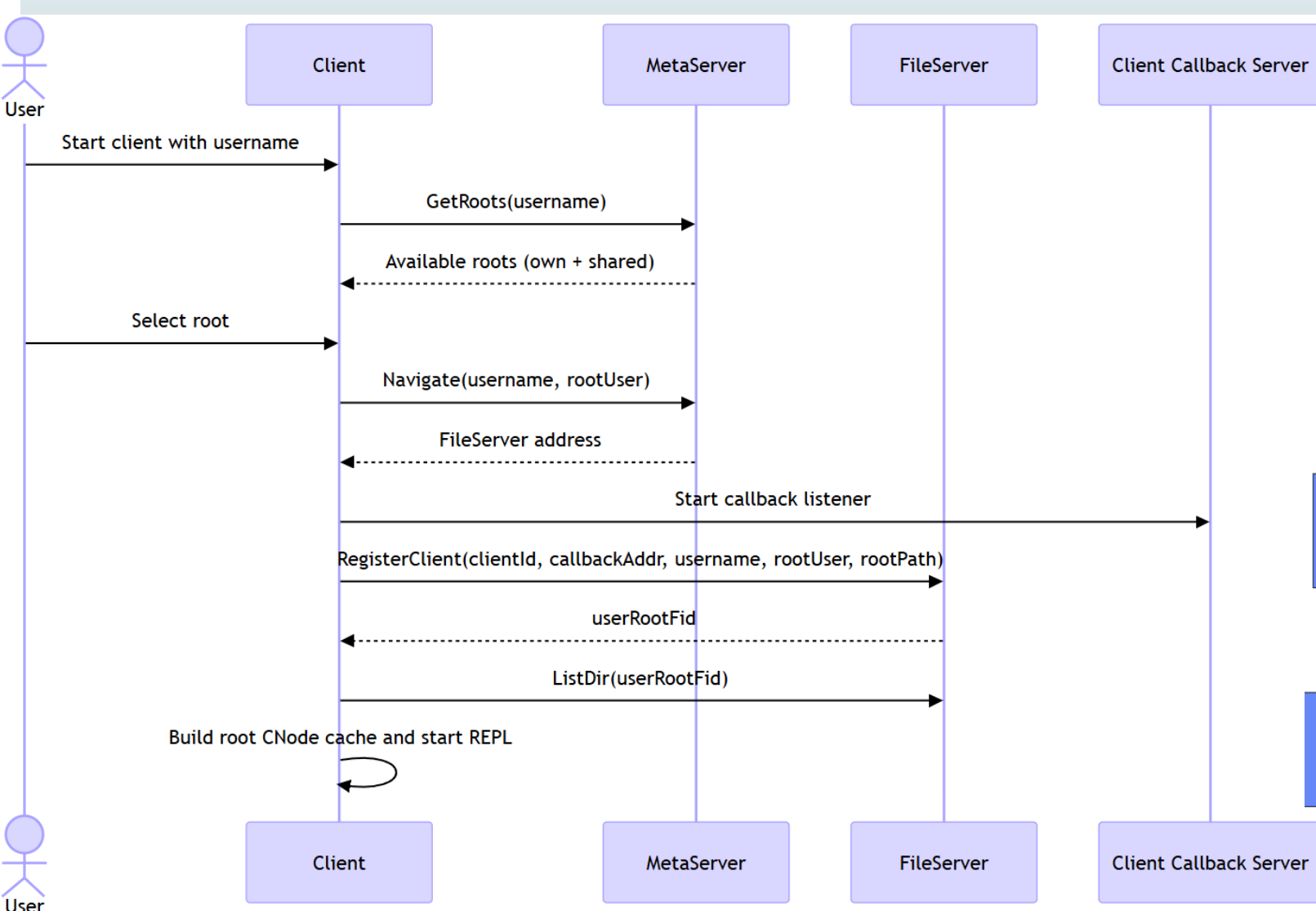
FID format

<FileServerID>_<InodeID>_<GenerationNumber> (example: fs1_7_1)

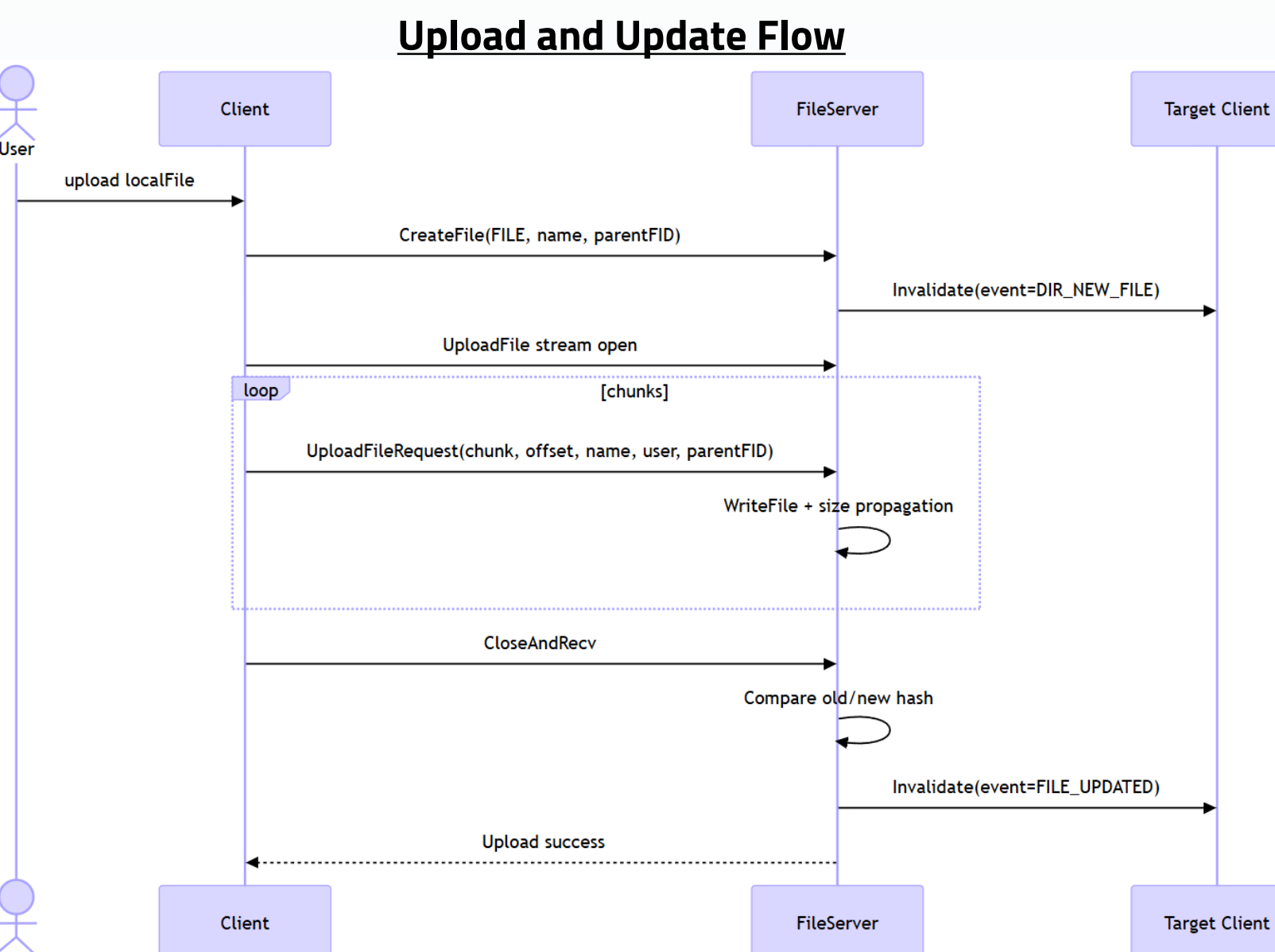
Session Initiation and Root Selection

Design Notes

- Root selection allows entering own root or shared roots from other owners.
- Registration enforces owner/shared ACL validation before granting root FID.
- MetaServer acts as the sole source of truth for all the FileServer's IPs and authorization engine for Shared directories.

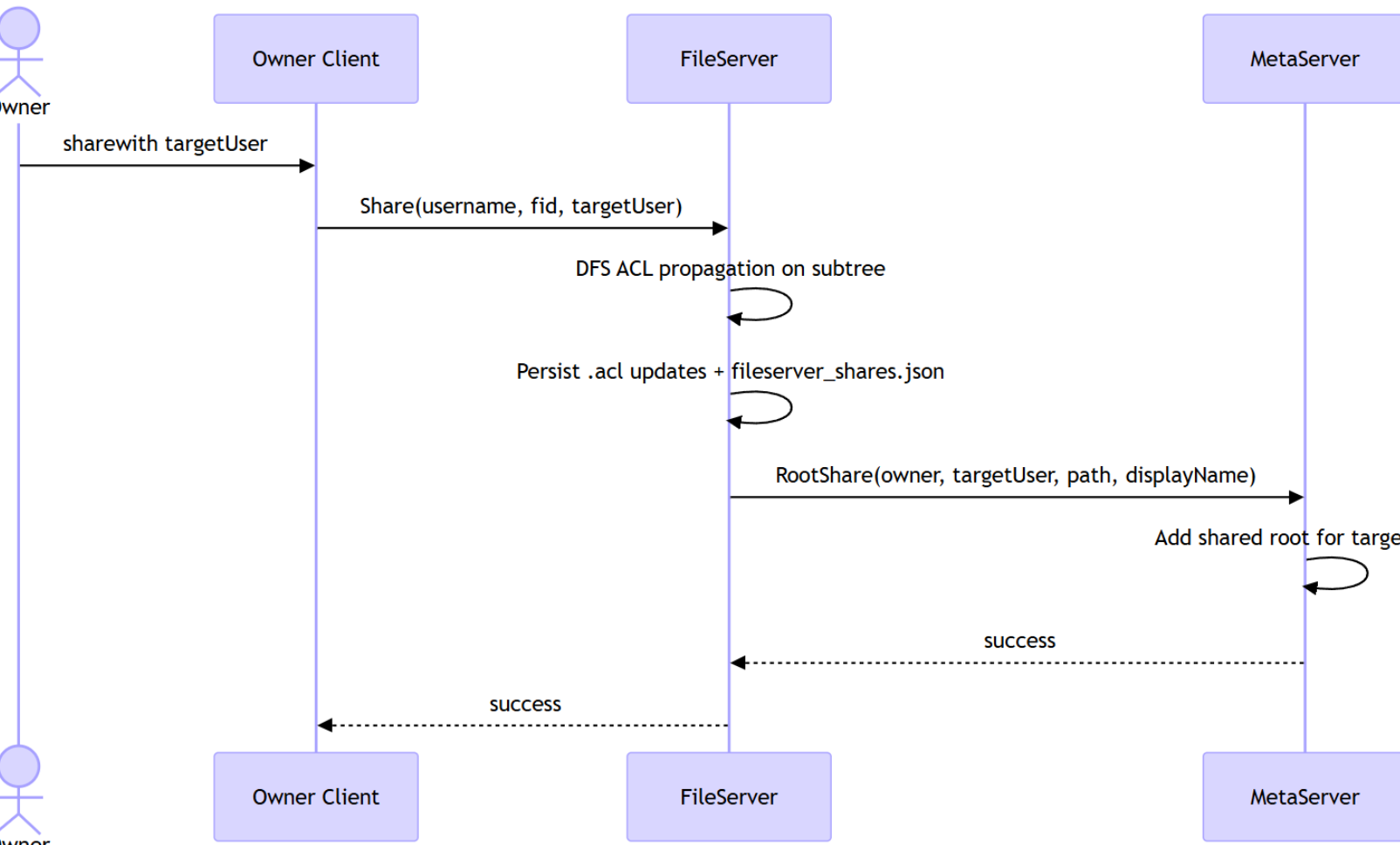


Protocol and Interaction Flows



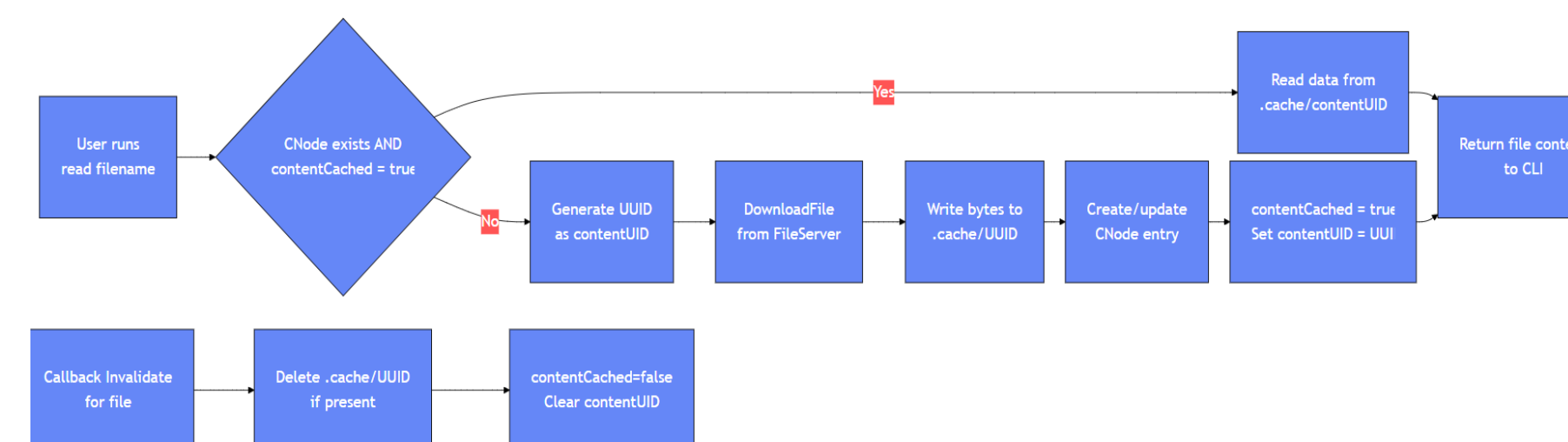
- Notify only **active sessions** (*lastSeenAt* ≤ 5 min).
- Notify only clients whose **currentDirFID** equals **event directory**

Share/Un-Share Flow



- Sharing is enforced in **two stages**: the FileServer propagates **ACL changes across the selected subtree** and **persists** them locally; **MetaServer updates global routing** so the target user can discover the shared root.
- This two-level update keeps both authorization (data plane) and discoverability (control plane) **consistent** across clients.

Caching Flow (Read Path)



- Cache files are stored in a temporary **.cache** local directory
- The cache is cleared as soon as the client shuts down

Command Coverage

CLI Command	Primary RPC(s)	Main Data Structures	Consistency / Side Effect
ls, pwd, viscache	none (cache/local path)	CNode tree	local-only inspection of metadata and cache structure
cd <path>, refresh	ChangeDir, ListDir	currentFID, current DirFID, CNode	updates session state and syncs directory view
info	GetAttr	inode metadata	read-only metadata fetch
read <name>	cache hit or DownloadFile	.cache/<UUID>, contentCached	cache fill on miss; invalidated by callback
create, mkdir, upload	CreateFile, UploadFile (stream)	inode tree, ACL inheritance, cache node	triggers new/update notification events
download <name>	ListDir, DownloadFile (stream)	local ./Download	server-to-client file transfer
rm, rm -r, trash, restore, show_trash, clear_trash	DeleteFile, TrashFile, RestoreFile, ShowTrash	inode graph, .trash, trashMeta	deletion lifecycle (soft + hard)
sharewith, unsharewith	Share, Unshare, RootShare, RootUnshare	ACL subtree + shared-root index	controls cross-user access and discovery
clear	none	terminal	UI-only command

Persistence, Reliability, and Security

Persistence

- Directory ACL stored per-directory in **.acl** JSON.
- Share index stored in **filesrvr_shares.json**.
- MetaServer state stored in **metaserver_state.json**.

Reliability

- Heartbeat** from FileServer to MetaServer.
- Stale FileServers excluded from navigation and allocation.
- Callback **timeout** and failure-based pruning.

Security

- ACL** owner/shared checks enforced **server-side** on operations.
- TLS** can be enabled for **all gRPC channels** (client-meta, client-file, file-callback).

Key Design Decisions

- Separate control plane** (MetaServer) and **data plane** (FileServer).
- Explicit **global IDs (FID)** for references across protocol boundaries.
- ACL based tracking** for robust and **secure** sharing of files/directories.
- Cache coherence** via callbacks + user-triggered refresh for list synchronization.
- Soft delete** (trash) with **metadata-assisted restore** for usability.

References

- "Operating systems: three easy pieces."**
<https://pages.cs.wisc.edu/~remzi/OSTEP/>
- Inspired by Google's **Drive** and Microsoft's **OneDrive**

We would like to thank Prof. Abhishek Bichhawat for his unwavering support and guidance, and Jaskirat Singh Maskeen and Shardul Junagade for their valued contributions throughout the project.